

Test Plan

Software Testing Final Report

Course: Software Testing

Task Management Platform

Team Members: Huang Yihe (2351269)
Guan Jingwen (2352744)
Huang Yihan (2351271)
Xia Haobo (2352048)

June 2026

Contents

1	Project Name and Tool	3
2	Project Scope	3
2.1	Background	3
2.2	Overall Objectives	3
2.3	Testing Scope	4
3	Test Items	4
3.1	Main Functional Test Items	4
3.2	Non-Functional Concerns	4
3.3	System Architecture and Main Components	5
4	High-Level Test Suite Design	6
4.1	Risk-Driven Strategy	6
4.2	Test Suite Planning	6
4.3	Reasons for Technique Selection	7
5	Schedule and Checklist	7
5.1	Tool-Assisted Testing Process	7
5.2	Manual Comparison Testing Process	8
5.3	Test Levels and Goals	8
5.4	Checklist	9
6	Organization Structure	9
7	Test Framework and Rationale	10
7.1	Selected Framework	10
7.2	Selection Rationale	10
7.3	Execution Commands	10

8	Cost Estimation	11
8.1	Tool-Assisted Testing Cost Estimation	11
8.2	Manual Comparison Testing Cost Estimation	11
8.3	Comparative Analysis	11
8.4	Cost Conclusion	12
9	Conclusion	12

1 Project Name and Tool

Project name: Task Management Platform

Testing tool: AutoTestDesign

2 Project Scope

2.1 Background

The system under test is a lightweight task management platform implemented with Flask and SQLite. It contains three main modules: user authentication, project management, and task management. The goal of this course project is not only to verify whether one API is usable, but also to build an automated test design tool and apply it to a real runnable target application. The project covers requirement analysis, risk analysis, test design, test execution, and result evaluation.

This test plan focuses on how to organize testing activities for the target application with AutoTestDesign, and how to combine tool-assisted testing with manual comparison testing. The later detailed test design and execution document uses the task management module, namely REQ-005, REQ-006, and REQ-007, as the concrete testing target.

2.2 Overall Objectives

Objective	Description
Verify the main functions of the target application	Check whether the major modules of the task management platform satisfy the requirements.
Verify tool usability	Evaluate whether AutoTestDesign is applicable to a real project.
Establish requirement-to-test traceability	Connect requirements, risks, testing techniques, and test cases.
Compare testing efficiency	Compare tool-assisted testing and manual testing in time cost and output.
Support course deliverables	Provide planning basis for risk analysis, detailed test design, and execution documents.

2.3 Testing Scope

Included scope:

- Functional requirement analysis and risk analysis of the target application
- Black-box test design and white-box test design
- Automatically generated test scripts and manually written comparison scripts
- Test result recording and defect summarization

Excluded scope:

- Dedicated frontend UI usability testing
- Performance stress testing and concurrency testing
- Penetration testing and deep security testing
- Production deployment and operation verification

3 Test Items

3.1 Main Functional Test Items

Module	Requirements	Main Test Content
User Authentication	REQ-001 to REQ-003	Registration, login, password reset
Project Management	REQ-004	Project creation, update, deletion, statistics
Task Management	REQ-005 to REQ-007	Task creation, update, deletion, assignment, statistics

3.2 Non-Functional Concerns

Although this project does not conduct large-scale non-functional testing separately, the test plan still pays attention to the following characteristics:

Characteristic	Focus
Reliability	Whether illegal inputs and abnormal paths are handled stably
Maintainability	Whether requirements, code, and test cases can be clearly mapped
Traceability	Whether risk analysis, test strategy, and execution results can be traced to each other
Usability	Whether the tool allows testers to interactively modify coverage items, strategies, and test cases

3.3 System Architecture and Main Components

The current system under test uses a layered backend structure:

```
flask_app/
|- app.py
|- config.py
|- routes/
|   |- auth.py
|   |- projects.py
|   `-- tasks.py
|- models/
|   |- database.py
|   |- project.py
|   `-- task.py
`- utils/
    `-- validators.py
```

The components directly related to testing are:

Component	File	Role
Authentication routes	flask_app/routes/auth.py	Registration, login, password reset
Project routes	flask_app/routes/projects.py	Project CRUD and statistics
Task routes	flask_app/routes/tasks.py	Task CRUD, assignment, query by user

Component	File	Role
Data models	flask_app/models/task.py, project.py	Task and project data operations
Validators	flask_app/utils/validators.py	Validation of title, status, priority, and other fields

4 High-Level Test Suite Design

4.1 Risk-Driven Strategy

According to the risk analysis, testing resources are allocated based on risk levels:

Risk Level	Requirements	Planned Test Strategy
High	REQ-001 to REQ-003	Prioritize authentication and account security scenarios, emphasizing input validation and process integrity.
Medium	REQ-004 to REQ-006	Use both black-box and white-box techniques to cover inputs, conditions, and key paths.
Low	REQ-007	Focus on statistical correctness and API consistency.

4.2 Test Suite Planning

This project plans to build test suites around five testing techniques:

Suite Type	Applicable Requirements	Technique	Design Goal
TS-BB-EP	Most requirements	Equivalence partitioning	Identify valid and invalid input partitions
TS-BB-BVA	Boundary-sensitive fields	Boundary value analysis	Cover lengths, ranges, and critical values

Suite Type	Applicable Requirements	Technique	Design Goal
TS-BB-DT	Condition-combination scenarios	Decision table	Cover major condition combinations and expected actions
TS-WB-PC	Key business logic	Path coverage	Cover important normal and abnormal return paths
TS-WB-ST	Requirements with lifecycle or state changes	State transition	Cover states and state transitions

4.3 Reasons for Technique Selection

Technique	Reason
Equivalence partitioning	The target system has many field validation scenarios.
Boundary value analysis	Title length, date, and status fields are suitable for boundary testing.
Decision table	Task creation, assignment, and statistics all contain clear condition combinations.
Path coverage	The route layer and model layer contain multiple normal and abnormal paths.
State transition	Task lifecycle and business status can be modeled as states.

5 Schedule and Checklist

5.1 Tool-Assisted Testing Process

This project organizes tool-assisted testing through a “one generator + one reviewer” collaboration model. The overall process is:

Phase	Activity	Role	Planned Output
P1	Import requirements and code repository	Generator	Standard requirements and requirement-code graph
P2	Risk analysis	Generator	Risk level, test priority, and risk factors
P3	Black-box test design	Generator	EP, BVA, DT analysis tables and test cases
P4	White-box test design	Generator	Path coverage and state transition results
P5	Manual interactive review	Reviewer	Modified coverage items, strategies, and test cases
P6	Export and execution	Generator and reviewer	Executable scripts and execution records

5.2 Manual Comparison Testing Process

To evaluate the tool effect, a manual comparison test is planned:

Phase	Activity	Participants	Planned Goal
M1	Read requirements and code manually	2 people	Independently understand the module under test
M2	Design test cases manually	2 people	Build a manual test set
M3	Write scripts manually	2 people	Produce manual scripts
M4	Compare results	2 people	Compare coverage, time cost, and defect discovery ability

5.3 Test Levels and Goals

Test Level	Object	Goal
Functional API testing	Target application APIs	Verify whether requirement behavior matches the specification
White-box structural testing	Key business code	Verify important paths and state transitions
Result review	Automatic and manual script outputs	Distinguish target application defects from script problems

5.4 Checklist

Check Item	Goal
Requirements have been standardized	Establish unified requirement descriptions
Risks have been classified	Assign risk levels and priorities to each requirement
At least three black-box techniques	Satisfy FR 3.0
At least one white-box technique	Satisfy the white-box testing requirement
Interactive review is supported	Testers can modify coverage items, strategies, and test cases
Structured artifacts can be exported	Support JSON, executable test scripts, and other outputs

6 Organization Structure

The organization of testing activities in this project is:

Test Lead

| - Generator

| ` - AutoTestDesign operation

| - Reviewer

| ` - Manual review and correction

` - Test scripts and result output

The responsibilities of each role are:

Role	Responsibility
Test Lead	Coordinate test scope, test strategy, and document output
Generator	Import requirements and code, run AutoTestDesign, and generate test assets
Reviewer	Review requirement mappings, risk analysis, test cases, and execution results

During the manual comparison testing phase, two members jointly participate in requirement review, test case design, and result comparison. This is used to judge the benefits and limitations of tool-assisted testing compared with manual testing.

7 Test Framework and Rationale

7.1 Selected Framework

This project selects **Pytest** as the execution framework.

7.2 Selection Rationale

Reason	Description
Same language as the target system	The target application is Python/Flask, and the testing framework is also Python.
Supports Flask <code>test_client()</code>	APIs can be executed directly inside the process.
Convenient automatic export	AutoTestDesign can directly output Pytest scripts.
Convenient result archiving	Command-line execution and failure reports are clear and suitable for a course project.

7.3 Execution Commands

```
python -m pytest test_scripts/test_generated.py -v --tb=short
python -m pytest test_scripts/test_generated_refined.py -v --tb=short
python -m pytest test_scripts/test_human.py -v --tb=short
```

8 Cost Estimation

8.1 Tool-Assisted Testing Cost Estimation

Using the task management module as the representative scenario, the estimated cost of tool-assisted testing is:

Item	Estimated Value
Input tokens	About 79,260
Output tokens	About 10,828
Model cost	About ¥2
Manual review time	About 1 hour
Overall tool-assisted process time	About 1 hour

8.2 Manual Comparison Testing Cost Estimation

Item	Estimated Value
Number of participants	2 people
Overall manual testing time	About 4 hours
Additional tool cost	0

8.3 Comparative Analysis

Metric	Tool-Assisted Testing	Manual Comparison Testing
Total time	About 1 hour	About 4 hours
Model or tool cost	About ¥2	0
Human participation mode	Generation + review	Full manual design and execution
Expected output	Risk analysis, analysis tables, test scripts, execution records	Manual scripts and manual result records

Note: The tool currently keeps the risk score, risk level, and test priority for each requirement. These fields are used for later sorting, filtering, and testing resource allocation.

8.4 Cost Conclusion

From the planning perspective, the main value of the automated test design tool is:

1. Quickly producing structured test assets.
2. Turning human work from “designing from scratch” into “reviewing and correcting” .
3. Shortening the test design stage with low model cost.

9 Conclusion

This test plan explains how AutoTestDesign is used to test the target application. It does not report concrete test results. It defines the test scope, test items, testing techniques, organization mode, execution framework, and cost estimation, and provides the overall plan for later detailed test design and execution.

The concrete execution results, script comparison, and defect discovery results related to this plan are discussed in the detailed test design and execution document.